

Fiche 311 — Le système objet S3 : classes, génériques, UseMethod et NextMethod

Matière	Code · Langage R
Cours source	R Core Team, <i>An Introduction to R</i> 4.6.1 — §10.9 « Classes, generic functions and object orientation » ; §3.4 « The class of an object »
Sources d'appoint	<i>R Language Definition</i> 4.6.1, chapitre 5 « Object-oriented programming » — §5.1 définition, §5.2 héritage, §5.3 dispatch, §5.4 UseMethod , §5.5 NextMethod , §5.6 méthodes de groupe, §5.7 écrire des méthodes ; §2.2.4 « Classes »
Difficulté	Avancé — le mécanisme qui explique le comportement de tout R
Temps d'étude estimé	1 h 30
Prérequis	Fiches 303, 306, 309, 310 (attributs, listes, fonctions, portée)
Concepts clés	attribut <code>class</code> , fonction générique , méthode <code>generic.class</code> , <code>UseMethod</code> , méthode par défaut , héritage par vecteur de classes, <code>NextMethod</code> , <code>.Generic</code> / <code>.Class</code> / <code>.Method</code> / <code>.Group</code> , méthodes de groupe <code>Math</code> , <code>Summary</code> , <code>Ops</code> , <code>methods()</code> , <code>getS3method()</code> , <code>getAnywhere()</code> , <code>unclass()</code> , S4 dans <code>methods</code>
À retenir en priorité	<code>generic.class</code> , puis la classe suivante, puis <code>generic.default</code> · <code>UseMethod</code> ne rend jamais la main · ne jamais nommer une fonction <code>gen.cl</code> par hasard · une classe se pose sans aucun contrôle.

Vue d'ensemble

LE PRINCIPE « The CLASS of an object determines HOW IT WILL BE TREATED by what are known as GENERIC FUNCTIONS. »

UNE GENERIQUE souvent UNE SEULE LIGNE :

```
mean
function (x, ...)
  UseMethod("mean")
```

LE DISPATCH `class(x) = c("foo", "bar")` et `generique = mean`

1. cherche `mean.foo`
2. sinon `mean.bar`
3. sinon `mean.default`
4. sinon ERREUR -> « always write a default method »

CE QUI EST IGNORE « The class attributes of ANY OTHER ARGUMENTS to mean ARE IGNORED. » -- seul le PREMIER argument compte

UseMethod ne CREE PAS un nouvel environnement : il reutilise celui de l'appel
« it DOES NOT RETURN CONTROL to the calling function »
-> tout ce qui suit UseMethod n'est JAMAIS execute

NextMethod passe a la CLASSE SUIVANTE du vecteur -- c'est l'HERITAGE

QUATRE VARIABLES SPECIALES posees dans le cadre : `.Generic .Class .Method .Group`

TROIS GROUPES

Math	<code>abs, cos, exp, log, round, ...</code>
Summary	<code>all, any, max, min, prod, range, sum</code>
Ops	<code>+ - * / ^ < > <= >= != == %% %/% & !</code>

EXPLORER

<code>methods(class = "data.frame")</code>	<code>methods(plot)</code>
<code>getS3method("coef", "aov")</code>	<code>getAnywhere("coef.aov")</code>
<code>unclass(x)</code>	pour voir dessous

Le problème posé. « *The class of an object determines how it will be treated by what are known as generic functions. Put the other way round, a generic function performs a task or action on its arguments specific to the class of the argument itself. If the argument lacks any class attribute, or has a class not catered for specifically by the generic function in question, there is always a default action provided.* » (§10.9)

INTUITION — C'EST CE QU'ON OBSERVE DEPUIS LA FICHE 300.

`plot(Expt, Speed)` produisait des boîtes à moustaches parce que `Expt` était un facteur ;
`plot(fr)` produisait un histogramme parce que `fr` était une "table" ; un data frame s'affiche en tableau et un facteur avec sa ligne `Levels:` . Une seule fonction, des comportements différents. **Ce chapitre en donne la mécanique.**

Le cadrage honnête de la *R Language Definition* (§5.1). « *Rather than having a full-fledged object-oriented system, R has a class system and a mechanism for dispatching based on the class of an object.* » Et l'avertissement de l'introduction du chapitre 5 : « *because of the way that the object system is incorporated into R this advantage [greater consistency] does not obtain. Users are cautioned to use the object system in a straightforward manner. While it is possible to perform some rather interesting feats these tend to lead to obfuscated code.* »

● Concept 1 — La classe, et l'absence totale de contrôle

Rappel (§3.4, fiche 303). « *All objects in R have a class, reported by the function `class`. For simple vectors this is just the mode ... but `"matrix"`, `"array"`, `"factor"` and `"data.frame"` are other possible values.* »

DÉFINITION (R LANGUAGE DEFINITION §5.1).

« *The class system is facilitated through the `class` attribute. This attribute is a character vector of class names. So to create an object of class `"foo"` one simply attaches a class attribute with the string `"foo"` in it. Thus, virtually anything can be turned into an object of class `"foo"`.* »

Il n'y a rien de plus. Pas de déclaration, pas de constructeur imposé, pas de vérification :

```
x <- 1:10
class(x) <- "foo"      # x est maintenant « de classe foo »
```

L'avertissement du §2.2.4 (fiche 303), qui prend ici tout son sens. « *This attribute **can be accessed and manipulated virtually without restriction by users. There is no checking that an object actually contains the components that class methods expect. Thus, altering the `class` attribute should be done with caution, and when they are available specific creation and coercion functions should be preferred.*** »

La conséquence est différée : poser une classe ne casse rien **immédiatement**. La panne survient **au premier appel générique**, quand une méthode cherche une composante qui n'existe pas — loin de la ligne fautive.

● Concept 2 — Une générique, et le dispatch

DÉFINITION (R LANGUAGE DEFINITION §5.1).

*« The object system makes use of generic functions **via two dispatching functions, `UseMethod` and `NextMethod`**. **The typical use of the object system is to begin by calling a generic function. This is typically a very simple function and consists of a single line of code.** »*

```
> mean  
function (x, ...)  
  UseMethod("mean")
```

Et l'exemple du cours (§10.9) :

```
> coef  
function (object, ...)  
  UseMethod("coef")
```

« **The presence of `UseMethod` indicates this is a generic function.** »

Règle — comment la méthode est choisie (§5.1). « When `mean` is called it can have any number of arguments **but its first argument is special and the class of that first argument is used to determine which method should be called.** ... **The class attributes of any other arguments to `mean` are ignored.** »

Règle — la recherche complète (§5.1). « Suppose that `x` had a class attribute that contained `"foo"` and `"bar"`, in that order. Then R would **first search for a function called `mean.foo`** ; if it did not find one it would then **search for a function `mean.bar`** ; and if that search was also unsuccessful then **a final search for `mean.default`** would be made. **If the last search is unsuccessful R reports an error. It is a good idea to always write a default method.** »

`class(x) = ("foo", "bar") ⇒ mean.foo → mean.bar → mean.default → erreur`

« **Note that the functions `mean.foo` etc. are referred to, in this context, as methods.** »

COMPLÉMENT (§5.3).

*« **Generic functions should consist of a single statement.** They should usually be of the form `foo <- function(x, ...) UseMethod("foo", x)` . When `UseMethod` is called, it determines the appropriate method and **that method is invoked with the same arguments, in the same order as the call to the generic, as if the call had been made directly to the method.** »*

« **If the first argument to the generic function has no class attribute then `generic.default` is used.** »

*« **Since the introduction of namespaces the methods may not be accessible by their names (i.e. `get("generic.class")` may fail), but they will be accessible by `getS3method("generic", "class")` .** »*

2.1 L'exemple du manuel : deux représentations d'un point

Exemple (§5.1). *« A point in two-dimensional Euclidean space can be specified by its **Cartesian** (x - y) or **polar** (r - θ) coordinates. Hence ... we could define two classes, "xypoint" and "rthetapoint". All xypoint data structures are **lists with an x-component and a y-component**. All rthetapoint objects are **lists with an r-component and a theta-component**. »*

```
xpos <- function(x, ...)
  UseMethod("xpos")

xpos.xypoint      <- function(x) x$x
xpos.rthetapoint <- function(x) x$r * cos(x$theta)
```

CONTRÔLE — VÉRIFIER LA SECONDE MÉTHODE.

En coordonnées polaires, un point de rayon r et d'angle θ a pour abscisse

$$x = r \cos \theta$$

— c'est exactement `x$r * cos(x$theta)`. Et la première méthode n'a rien à calculer : l'abscisse **est** la composante `x`.

La leçon (§5.1). « *The user simply calls the function `xpos` with either representation as the argument. The internal dispatching method finds the class of the object and calls the appropriate methods.* »

« *It is pretty easy to add other representations. One need not write a new generic function, only the methods. This makes it easy to add to existing systems since the user is only responsible for dealing with the new representation and not with any of the existing representations.* »

C'est l'argument central du système. Ajouter une classe **n'oblige à toucher à rien** de ce qui existe. C'est l'inverse d'un `switch` sur un type, qu'il faudrait modifier à chaque ajout.

« *The bulk of the uses of this methodology are to provide specialized printing for objects of different types ; there are about 40 methods for `print`.* »

● Concept 3 — `UseMethod` : deux ruptures avec l'évaluation ordinaire

DÉFINITION (§5.4).

« `UseMethod` is a special function and it behaves differently from other function calls. The syntax is `UseMethod(generic, object)` ... `UseMethod` can only be called from the body of a function. »

Règle — les deux changements (§5.4). « `UseMethod` changes the evaluation model in two ways. »

1. « when it is invoked it determines the next method (function) to be called. It then invokes that function using the current evaluation environment. » — « Rather than creating a new evaluation environment `R` uses the environment of the current function call (the call to the generic). ... The arguments that were used in the call to the generic are rematched to the formal arguments of the selected method. »
2. « it does not return control to the calling function. This means that any statements after a call to `UseMethod` are guaranteed not to be executed. »

La seconde règle est la plus concrète : du code écrit après `UseMethod` dans le corps d'une générique est du code mort. C'est pourquoi « generic functions should consist of a single statement ».

Les quatre variables spéciales (§5.4). *« The call to `UseMethod` has the effect of placing some special objects in the evaluation frame. They are `.Class`, `.Generic` and `.Method`. ... `.Class` is the class of the object, `.Generic` is the name of the generic function and `.Method` is the name of the method currently being invoked. If the method was invoked through one of the internal interfaces then there may also be an object called `.Group`. »*

« After the initial call to `UseMethod` these special variables, not the object itself, control the selection of subsequent methods. » — c'est ce qui rend `NextMethod` possible (concept 4).

Deux précisions utiles (§5.4).

- « *If the first argument to `UseMethod` is not supplied it is assumed to be the name of the current function.* »
- *« If **two arguments** are supplied ... **the first is the name of the method and the second is assumed to be the object that will be dispatched on.** It is evaluated so that the required method can be determined. **In this case the first argument in the call to the generic is not evaluated and is discarded. There is no way to change the other arguments in the call to the method ; this is in contrast to `NextMethod` , where the arguments can be altered.** »*
- « *Any arguments to the generic that were evaluated prior to the call to `UseMethod` remain evaluated.* »
- « *[Prior to R 4.4.0 any local assignments in the frame of the generic would be carried forward into the call to the method ; this is no longer the case.]* »

● Concept 4 — `NextMethod` : l'héritage

DÉFINITION (§5.5).

« `NextMethod` is used to provide a simple inheritance mechanism. »

Définition (§5.2). « *The class attribute of an object can have several elements. ... inheritance is mainly handled through `NextMethod` . `NextMethod` determines the method currently being evaluated, finds the next class from the class attribute, and proceeds.* »

Le patron d'usage (§5.1). « *Methods themselves employ `NextMethod` to provide a form of inheritance. Commonly a specific method performs a few operations to set up the data and then it calls the next appropriate method through a call to `NextMethod` .* »

Le sens du vecteur de classes devient clair : `class(x) <- c("specifique", "general")` signifie « traite-moi d'abord comme un `specifique` , et pour le reste comme un `general` ». La méthode spécifique fait ce qu'elle a de particulier, puis passe la main.

Règle — la syntaxe et les défauts (§5.5). *« The syntax for a call to `NextMethod` is `NextMethod(generic, object, ...)` . If the generic is not supplied the value of `.Generic` is used. If the object is not supplied the first argument in the call to the current method is used. Values in the `...` argument are used to modify the arguments of the next method. »*

Règle — comment les arguments sont transmis (§5.5). « *Methods invoked as a result of a call to `NextMethod` behave as if they had been invoked from the previous method. The arguments to the inherited method are in the same order and have the same names as the call to the current method ... the expressions for the arguments are the names of the corresponding formal arguments of the current method. Thus the arguments will have values that correspond to their value at the time `NextMethod` was invoked.* »

« *Unevaluated arguments remain unevaluated. Missing arguments remain missing.* » — l'évaluation paresseuse de la fiche 309 traverse le dispatch **sans être forcée**.

Le piège de l'appel direct (§5.1 et §5.7). *« A function may have a call to `NextMethod` anywhere in it. The determination of which method should then be invoked is based primarily on the current values of `.Class` and `.Generic` . This is somewhat problematic since the method is really an ordinary function and users may call it directly. If they do so then there will be no values for `.Generic` or `.Class` . »*

« *If a method is invoked directly and it contains a call to `NextMethod` then the first argument to `NextMethod` is used to determine the generic function. An error is signalled if this argument has not been supplied ; it is therefore a good idea to always supply this argument.* »

« *In the case that a method is invoked directly the class attribute of the first argument to the method is used as the value of `.Class` .* »

Règle pratique : quand une méthode contient un `NextMethod` , écrire le générique explicitement — `NextMethod("mean")` plutôt que `NextMethod()` .

Énoncé. Créer une classe `"celsius"` pour des températures, avec un affichage qui ajoute l'unité, tout en conservant le comportement numérique ordinaire pour le reste.

Étape 1 — poser la classe. Rien de plus qu'un attribut (§5.1) — « *to create an object of class "foo"* **one simply attaches a class attribute** ». On met **deux** classes, du plus spécifique au plus général :

```
celsius <- function(x) {  
  class(x) <- c("celsius", "numeric")  
  x  
}  
t <- celsius(c(18.5, 21.0, 3.2))
```

Étape 2 — écrire la méthode d'affichage. Le générique est `print`, la classe est `celsius`, donc la méthode s'appelle `print.celsius` — c'est la règle `generic.class` du §5.3.

```
print.celsius <- function(x, ...) {  
  cat(paste0(unclass(x), " °C"), "\n")  
  invisible(x)  
}
```

`unclass(x)` **est indispensable ici**. Sans lui, `paste0` déclencherait à nouveau... non : `paste0` n'est pas générique, mais tout appel qui **réafficherait** `x` relancerait `print.celsius` — récursion infinie. `unclass()` (§3.4, fiche 303) retire l'effet de classe : c'est **l'usage exact** que le cours lui donne.

`invisible(x)` : une méthode `print` doit rendre son argument **de façon invisible**, sinon l'affichage se dédouble. C'est le « *unless specifically made invisible* » du §1.8 (fiche 300).

Étape 3 — vérifier le dispatch. Taper `t` appelle `print`, qui est générique. `.Class` vaut `c("celsius", "numeric")`. R cherche `print.celsius` → trouvé. Affichage : `18.5 °C 21 °C 3.2 °C`.

Étape 4 — ce qui se passe pour les autres génériques. `mean(t)` : R cherche `mean.celsius` → absent ; puis `mean.numeric` → absent ; puis `mean.default` → trouvé. Le calcul se fait normalement. **C'est le rôle de la méthode par défaut**, et la raison pour laquelle le manuel écrit « *it is a good idea to always write a default method* ».

Étape 5 — ajouter une méthode qui hérite. Supposons qu'on veuille un `summary` qui annonce l'unité puis fasse le résumé habituel :

```
summary.celsius <- function(object, ...) {
  cat("Températures en degrés Celsius\n")
  NextMethod("summary")
}
```

`NextMethod("summary")` « *determines the method currently being evaluated, **finds the next class from the class attribute**, and proceeds* » : après `celsius` vient `numeric`, puis le défaut. **La méthode spécifique fait son ajout, et délègue le reste** — c'est exactement le patron du §5.1.

Étape 6 — pourquoi écrire `"summary"` explicitement. Parce que si quelqu'un appelle `summary.celsius(t)` **directement**, « *there will be no values for `.Generic` or `.Class`* » et « ***an error is signalled if this argument has not been supplied*** ». Le donner coûte neuf caractères et supprime la panne.

Étape 7 — vérifier ce qui existe. `methods(class = "celsius")` liste les deux méthodes écrites. `methods(summary)` montre combien de classes `summary` sait traiter.

Étape 8 — la mise en garde. Rien n'a vérifié que `t` contenait bien des nombres. « ***There is no checking that an object actually contains the components that class methods expect.*** » Un `class(x) <- "celsius"` sur une chaîne produirait un objet parfaitement légal et parfaitement absurde.

● Concept 5 — Les méthodes de groupe

DÉFINITION (§5.6).

« *For several types of internal functions **R provides a dispatching mechanism for operators**. This means that **operators such as `==` or `<`** can have their behaviour modified for members of **special classes**. The functions and operators have been grouped into **three categories** ... **There is currently no mechanism to add groups. It is possible to write methods specific to any function within a group.*** »

Groupe	Fonctions
Math	abs , acos , acosh , asin , asinh , atan , atanh , ceiling , cos , cosh , cospi , cumsum , exp , floor , gamma , lgamma , log , log10 , round , signif , sin , sinh , sinpi , tan , tanh , tanpi , trunc
Summary	all , any , max , min , prod , range , sum
Ops	+ - * / ^ < > <= >= != == %% %/% & !

La règle de résolution du groupe Ops (§5.6). « For operators in the `Ops` group **a special method is invoked if the two operands taken together suggest a single method.** Specifically, **if both operands correspond to the same method, or if one operand corresponds to a method that takes precedence over that of the other operand. If they do not suggest a single method then the default method is used. Either a group method or a class method dominates if the other operand has no corresponding method. A class method dominates a group method.** »

Deux règles de priorité à retenir : (1) **une méthode de classe l'emporte sur une méthode de groupe ;** (2) **si les deux opérandes ne s'accordent pas sur une méthode unique, c'est le défaut.**

Le cas particulier de `.Method` (§5.6). « When the group is `Ops` **the special variable `.Method` is a character vector with two elements.** The elements are set to **the name of the method if the corresponding argument is a member of the class that was used to determine the method.** Otherwise the corresponding element of `.Method` is set to the zero length string, `""`. »

C'est ce qui permet à une méthode d'opérateur de savoir **lequel des deux opérandes** a déclenché le dispatch — information indispensable pour un opérateur **non commutatif**.

● Concept 6 — Explorer le système

MÉTHODE (§10.9).

« *The number of generic functions that can treat a class in a specific way can be quite large. For example the functions that can accommodate objects of class "data.frame" include `[`, `[[<-`, `any`, `as.matrix`, `[<-`, `mean`, `plot`, `summary`. A currently complete list can be got by using the `methods()` function* » :

```
methods(class = "data.frame")
```

« *Conversely the number of classes a generic function can handle can also be quite large. For example the `plot()` function has a default method and variants for objects of classes "data.frame", "density", "factor", and more* » :

```
methods(plot)
```

Deux questions symétriques, deux appels : « *que sait-on faire de cette classe ?* » → `methods(class = ...)` ; « *quelles classes cette générique sait-elle traiter ?* » → `methods(generique)` .

MÉTHODE — LIRE LE CODE D'UNE MÉTHODE INVISIBLE (§10.9).

« In this example there are six methods, none of which can be seen by typing its name (as indicated by the asterisk). We can read these by either of » :

```
getAnywhere("coef.aov")
getS3method("coef", "aov")
```

Le cours en donne le résultat :

```
function (object, complete = FALSE, ...)
{
  cf <- object$coefficients
  if (complete)
    cf
```

```
else cf[!is.na(cf)]
}
```

L'astérisque dans la sortie de `methods()` signale une méthode non exportée — conséquence des espaces de noms (fiche 310) : « *objects cannot be masked ...* », mais ils ne sont pas non plus visibles depuis l'extérieur. `getS3method` est la voie officielle.

La règle de nommage à ne jamais oublier (§10.9). « *A function named `gen.cl` will be invoked by the generic `gen` for class `cl`, so do not name functions in this style unless they are intended to be methods.* »

C'est un piège réel et courant. Une fonction innocemment nommée `plot.data` sera prise pour la méthode `plot` de la classe `data`. Les points étant permis dans les noms R (fiche 300), rien ne distingue `summary.stats` d'une méthode. Utiliser `_` ou la casse pour les noms composés qui ne sont pas des méthodes.

« *The reader is referred to **The R Language Definition** for a more complete discussion of this mechanism.* »

Rappel (§3.4, fiche 303). `unclass()` « *removes temporarily the effects of class* » — l'outil d'exploration : il montre qu'un data frame **est une liste**, qu'un facteur **est un vecteur d'entiers**.

Et la note 4 du §3.4 : « *A different style using "formal" or "S4" classes is provided in package `methods`.* » — un second système objet, non traité ici.

Comment reconnaître le type d'exercice

Le signal dans l'énoncé	Ce qu'il faut mobiliser
« la même fonction se comporte autrement »	dispatch sur la classe du premier argument
« le corps de la fonction fait une ligne »	c'est une générique — chercher <code>UseMethod</code>
« quelle méthode est appelée ? »	<code>generic.classe1</code> → <code>generic.classe2</code> → <code>generic.default</code>
« erreur : pas de méthode applicable »	aucun <code>.default</code> n'a été écrit
« la classe du second argument est ignorée »	c'est la règle — seul le premier compte
« ajouter un nouveau type sans toucher au reste »	écrire une méthode , pas une générique
« faire un peu plus, puis le traitement normal »	<code>NextMethod()</code>
« erreur en appelant la méthode directement »	<code>.Generic</code> absent → nommer le générique
« redéfinir <code>+</code> pour ma classe »	méthode de groupe <code>Ops</code> , ou méthode de classe
« quelles fonctions connaissent ma classe ? »	<code>methods(class = "...")</code>
« quelles classes cette fonction traite-t-elle ? »	<code>methods(générique)</code>
« la méthode a un astérisque, je ne peux pas la lire »	<code>getS3method()</code> ou <code>getAnywhere()</code>
« ma fonction est appelée à ma place »	son nom ressemble à <code>générique.classe</code>
« voir ce qu'il y a vraiment dans l'objet »	<code>unclass()</code>
« et S4 ? »	le paquet <code>methods</code> — autre système

Comment résoudre ce type d'exercice

Protocole « créer une classe S3 » — 5 étapes.

1. Choisir **la structure de données** — le plus souvent **une liste** (fiche 306).
2. Écrire **une fonction de construction** qui pose la classe : `class(x) <- c("spécifique", "general")`, **du plus spécifique au plus général**.

3. Écrire au minimum `print.maclasse` — « *the bulk of the uses ... are to provide specialized printing* » — en pensant à `unclass()` et `invisible()` .
4. Pour les comportements partagés, écrire une **générique** d'une seule ligne `UseMethod("nom")` et **toujours une** `.default` .
5. Vérifier avec `methods(class = "maclasse")` .

Protocole « comprendre quelle méthode s'exécute » — 4 étapes.

1. `class(x)` — le vecteur, **dans l'ordre**.
2. Parcourir `generic.classe1` , `generic.classe2` , ... puis `generic.default` .
3. Vérifier que la méthode existe : `methods(generique)` ; la lire par `getS3method()` .
4. Si un `NextMethod` intervient, **reprendre le vecteur de classes à partir de la classe courante**.

Protocole « écrire une méthode sûre » — 4 étapes.

1. La nommer **exactement** `generique.classe` .
2. **Reprendre la signature de la générique** — les arguments sont **réappariés** aux formels de la méthode.
3. Si elle contient un `NextMethod` , **passer le générique en argument**.
4. Ne pas écrire de code **après** un `UseMethod` : « *guaranteed not to be executed* ».

● Common mistakes

Erreur	Correction
Croire que poser une classe convertit l'objet	« <i>there is no checking</i> » — la panne est différée
Attendre le dispatch sur le second argument	« <i>the class attributes of any other arguments ... are ignored</i> »
Omettre la méthode <code>.default</code>	« <i>it is a good idea to always write a default method</i> »
Écrire du code après <code>UseMethod</code>	« <i>guaranteed not to be executed</i> »
Écrire une générique de plusieurs instructions	« <i>should consist of a single statement</i> »
Appeler <code>NextMethod()</code> sans argument dans une méthode appelée directement	erreur — nommer le générique
Croire les arguments réévalués par <code>NextMethod</code>	« <i>unevaluated arguments remain unevaluated</i> »
Croire qu' <code>UseMethod</code> crée un nouvel environnement	il réutilise celui de l'appel à la générique
Nommer une fonction <code>plot.data</code> sans intention de méthode	elle deviendra une méthode
Chercher une méthode par <code>get("gen.c1")</code>	depuis les espaces de noms → <code>getS3method()</code>
Oublier <code>unclass()</code> dans <code>print.maclasse</code>	réursion ou affichage inutilisable
Oublier <code>invisible()</code> dans une méthode <code>print</code>	affichage dédoublé
Croire qu'on peut ajouter un groupe	« <i>There is currently no mechanism to add groups.</i> »
Croire qu'une méthode de groupe prime	« <i>a class method dominates a group method</i> »
Compter sur les assignations locales de la générique	plus transmises depuis R 4.4.0
Confondre S3 et S4	S4 vit dans le paquet <code>methods</code>

Ultimate Review

Le principe. « *The class of an object determines how it will be treated by ... generic functions. ... If the argument lacks any class attribute, or has a class not catered for ... there is always a default action provided.* »

Ce que R a, et ce qu'il n'a pas. « *Rather than having a full-fledged object-oriented system, R has a class system and a mechanism for dispatching based on the class of an object.* » Quatre objets spéciaux dans le cadre : `.Generic` , `.Class` , `.Method` , `.Group` .

La classe. « *a character vector of class names* » · « *to create an object of class "foo" one simply attaches a class attribute* » · « *virtually anything can be turned into an object of class "foo"* », et « *there is no checking* ».

Une générique. « *typically a very simple function and consists of a single line of code* » : `mean <- function(x, ...) UseMethod("mean")` · « *The presence of UseMethod indicates this is a generic function.* » · forme recommandée : `foo <- function(x, ...) UseMethod("foo", x)` .

Le dispatch. Seul le premier argument compte — « *the class attributes of any other arguments ... are ignored* » · pour `class(x) = c("foo", "bar")` : `mean.foo` → `mean.bar` → `mean.default` → **erreur** · « *it is a good idea to always write a default method* » · sans attribut de classe, `generic.default` · avec les espaces de noms, `get("gen.cl")` peut échouer → `getS3method("gen", "cl")` .

UseMethod — deux ruptures. (1) « *Rather than creating a new evaluation environment R uses the environment of the current function call* », et « *the arguments ... are rematched to the formal arguments of the selected method* » ; (2) « *it does not return control to the calling function ... any statements after a call to UseMethod are guaranteed not to be executed* ». Sans premier argument, il prend le nom de la fonction courante ; avec deux, le second est l'objet de dispatch et le premier argument de la générique **n'est pas évalué et est écarté**. Les arguments déjà évalués **le restent**. (Depuis R 4.4.0, les assignations locales de la générique **ne sont plus transmises**.)

NextMethod — l'héritage. « *determines the method currently being evaluated, finds the next class from the class attribute, and proceeds* » · patron : « *a specific method performs a few operations to set up the data and then calls the next appropriate method* » · syntaxe `NextMethod(generic, object, ...)` ; sans générique, `.Generic` ; sans objet, **le premier argument** ; le `...` sert à **modifier les arguments** du suivant · « *Unevaluated arguments remain unevaluated. Missing arguments remain missing.* » · appel direct d'une méthode → **pas de** `.Generic` → « *an error is signalled if this argument has not been supplied* ».

Méthodes de groupe. Trois groupes, **non extensibles** : `Math` (fonctions mathématiques), `Summary` (`all` any max min prod range sum), `Ops` (opérateurs arithmétiques, de comparaison et logiques) · « *a special method is invoked if the two operands taken together suggest a single method* » ; sinon **le défaut** · une

méthode de classe domine une méthode de groupe · pour `Ops` , `.Method` **a deux éléments**, `" "` pour l'opérande qui n'a pas déclenché le dispatch.

Explorer. `methods(class = "data.frame")` — ce qu'on sait faire de la classe · `methods(plot)` — ce que la générique sait traiter · `getS3method("coef", "aov")` ou `getAnywhere("coef.aov")` — lire une méthode marquée d'un **astérisque** · `unclass()` — voir dessous.

La règle de nommage. « *A function named `gen.cl` will be invoked by the generic `gen` for class `cl` , so do not name functions in this style unless they are intended to be methods.* »

Et au-delà. *« The bulk of the uses of this methodology are to **provide specialized printing** ... there are **about 40 methods for** `print` . »* · S4 : « a different style using **"formal" or "S4" classes** is provided in package `methods` ».

Active Recall

« When `mean` is called it can have any number of arguments **but its first argument is special and the class of that first argument is used to determine which method should be called ... The class attributes of any other arguments to `mean` are ignored.** » (§5.1)

*« Suppose that `x` had a class attribute that contained `"foo"` and `"bar"` , in that order. Then R would first search for `mean.foo` ; if it did not find one, `mean.bar` ; and if that search was also unsuccessful, `mean.default` . **If the last search is unsuccessful R reports an error.** »*

`mean.foo` → `mean.bar` → `mean.default` → erreur

« **It is a good idea to always write a default method.** » Et « if the first argument ... **has no class attribute** then `generic.default` is used ».

À la présence de `UseMethod` . Le cours le dit littéralement : « **The presence of `UseMethod` indicates this is a generic function.** » (§10.9)

```
> coef
function (object, ...)
UseMethod("coef")

> mean
function (x, ...)
UseMethod("mean")
```

« *This is typically **a very simple function** and consists of **a single line of code*** » (§5.1), et la forme recommandée est `foo <- function(x, ...) UseMethod("foo", x)` (§5.3).

Taper le nom d'une fonction sans parenthèses suffit donc à savoir si elle est générique — c'est le « *most of the system supplied functions are themselves written in R* » de la fiche 300 mis à profit.

« `UseMethod` **changes the evaluation model in two ways.** » (§5.4)

1. « *when it is invoked it **determines the next method to be called**. It then **invokes that function using the current evaluation environment*** » — et plus loin : « ***Rather than creating a new evaluation environment R uses the environment of the current function call (the call to the generic).** ... The arguments ... **are rematched to the formal arguments of the selected method.*** »

2. « *it **does not return control to the calling function**. This means that **any statements after a call to** `UseMethod` **are guaranteed not to be executed.*** »

La seconde a une conséquence directe : du code écrit après `UseMethod` est **du code mort**. D'où « *generic functions **should consist of a single statement*** ».

Par **le vecteur de classes** et `NextMethod` .

« *The class attribute of an object can have several elements ... inheritance is mainly handled through `NextMethod` . `NextMethod` **determines the method currently being evaluated, finds the next class from the class attribute, and proceeds.*** » (§5.2)

Le patron d'usage (§5.1) : « ***Commonly a specific method performs a few operations to set up the data and then it calls the next appropriate method through a call to***

`NextMethod` . »

Ainsi `class(x) <- c("specifique", "general")` se lit : « traite-moi d'abord comme un `specifique` , puis comme un `general` ». **L'ordre du vecteur est l'ordre d'héritage.**

« *Unevaluated arguments remain unevaluated. Missing arguments remain missing.* » — la paresse (fiche 309) traverse le dispatch.

Parce qu'une méthode **peut être appelée directement**, hors du dispatch.

« *This is somewhat problematic since **the method is really an ordinary function and users may call it directly.** If they do so then there will be no values for `.Generic` or `.Class`.* » (§5.1)

« *If a method is invoked directly and it contains a call to `NextMethod` then the first argument to `NextMethod` is used to determine the generic function. An error is signalled if this argument has not been supplied ; it is therefore a good idea to always supply this argument.* »

Et : « *In the case that a method is invoked directly **the class attribute of the first argument to the method is used as the value of `.Class`*** » — donc `.Class` se reconstitue, mais **pas `.Generic`** .

Neuf caractères contre une classe entière de pannes.

*« A point in two-dimensional Euclidean space can be specified by its **Cartesian (x - y)** or **polar (r - θ)** coordinates ... we could define two classes, `"xypoint"` and `"rthetapoint"` . »* (§5.1)

```
xpos <- function(x, ...)
  UseMethod("xpos")

xpos.xypoint      <- function(x) x$x
xpos.rthetapoint <- function(x) x$r * cos(x$theta)
```

La seconde méthode est $x = r \cos \theta$ — la conversion polaire → cartésienne.

« *The user simply calls the function `xpos` with either representation as the argument. The internal dispatching method finds the class of the object and calls the appropriate methods.* »

L'argument décisif : « *It is pretty easy to add other representations. One need not write a new generic function, only the methods ... the user is only responsible for dealing with the new representation and not with any of the existing representations.* » C'est l'inverse d'un `switch` sur un type, qu'il faudrait rouvrir à chaque ajout.

« *The functions and operators have been grouped into **three categories** ... There is currently no mechanism to add groups.* » (§5.6)

Groupe	Contenu
Math	<code>abs</code> , <code>cos</code> , <code>exp</code> , <code>log</code> , <code>round</code> , <code>signif</code> , <code>trunc</code> , <code>cumsum</code> ...
Summary	<code>all</code> , <code>any</code> , <code>max</code> , <code>min</code> , <code>prod</code> , <code>range</code> , <code>sum</code>
Ops	les opérateurs arithmétiques, de comparaison et logiques

La règle de `Ops` : « *a special method is invoked if the two operands taken together suggest a single method ... If they do not suggest a single method then the default method is used. Either a group method or a class method dominates if the other operand has no corresponding method. A class method dominates a group method.* »

Et « *when the group is `Ops` `.Method` is a character vector with two elements ... set to ""* » pour l'opérande qui n'a pas déterminé la méthode — indispensable pour un opérateur **non commutatif**.

Deux questions symétriques (§10.9) :

```
methods(class = "data.frame") # ce qu'on sait faire de cette classe
```

```
methods(plot)
```

```
# ce que cette generique sait traiter
```

*« In this example there are six methods, **none of which can be seen by typing its name (as indicated by the asterisk). We can read these by either of** »* :

```
getAnywhere("coef.aov")  
getS3method("coef", "aov")
```

L'astérisque signale une méthode non exportée — conséquence des espaces de noms (fiche 310). Le §5.3 le précise : « *Since the introduction of namespaces the methods may not be accessible by their names (i.e. `get("generic.class")` may fail), but they will be accessible by `getS3method("generic", "class")` ».*

« *A function named `gen.cl` will be invoked by the generic `gen` for class `cl`, so do not name functions in this style unless they are intended to be methods.* » (§10.9)

Le piège vient de la syntaxe des noms R : le point est un caractère **ordinaire** dans un identifiant (fiche 300). Rien ne distingue donc `plot.data`, choisi comme « le tracé des données », de **la méthode `plot` de la classe `data`**.

Le jour où un objet acquiert la classe `"data"` — volontairement ou par une bibliothèque tierce —, **votre fonction est appelée à la place de `plot`**, avec des arguments qu'elle n'attend pas.

Le remède : réserver le point aux méthodes, et utiliser `_` ou la casse pour les autres noms composés.

Rien — jusqu'au premier appel générique.

« *This attribute can be accessed and manipulated virtually without restriction by users. There is no checking that an object actually contains the components that class methods expect. Thus, altering the `class` attribute should be done with caution, and when they are available specific creation and coercion functions should be preferred.* » (§2.2.4)

Et le §5.1 le dit sans détour : « *virtually anything can be turned into an object of class `"foo"`* ».

La panne est donc différée et déplacée : elle survient quand une méthode cherche `object$coefficients` sur un objet qui n'en a pas, avec un message qui ne mentionne ni la classe ni la ligne où elle a été posée.

Le remède : une **fonction de construction** qui valide, plutôt qu'un `class(x) <- ...` nu.

Parce que la méthode reçoit un objet **qui porte encore sa classe**. Toute opération qui provoquerait un nouvel affichage relancerait **la même méthode** — récursion sans fin.

`unclass()` « *removes **temporarily** the effects of class* » (§3.4, fiche 303). Le cours en fait explicitement **un outil d'apprentissage** : « *one [situation] is when you are **learning to come to terms with the idea of class and generic functions*** ».

Et une méthode `print` doit rendre son argument **invisiblement** — sinon l'affichage se dédouble, puisque la valeur renvoyée serait à son tour imprimée (fiche 300, §1.8 : « *printed **unless specifically made invisible*** »).

Le squelette sûr : faire l'affichage à partir d' `unclass(x)` , terminer par `invisible(x)` .

« ***Rather than having a full-fledged object-oriented system, R has a class system and a mechanism for dispatching based on the class of an object.*** » (§5.1)

Et l'introduction du chapitre 5 est plus explicite encore : « *One of the advantages that most object systems impart is **greater consistency**. This is achieved via the rules that are checked by the compiler or interpreter. **Unfortunately because of the way that the object system is incorporated into R this advantage does not obtain.*** »

La recommandation : « *Users are cautioned to use the object system in a **straightforward manner**. While it is possible to perform some rather interesting feats **these tend to lead to obfuscated code and may depend on implementation details that will not be carried [forward]**.* »

Autrement dit : **s'en tenir au patron simple** — une classe, des méthodes nommées `generique.classe` , une `.default` , un `NextMethod` explicite quand il faut hériter.

Flashcards

Question	Réponse
Que détermine la classe d'un objet ?	Comment les fonctions génériques le traitent
Qu'est-ce qu'une fonction générique ?	Une fonction qui agit selon la classe de son argument
À quoi la reconnaît-on ?	À la présence de <code>UseMethod</code>
De combien de lignes est-elle faite ?	Une seule , en général
La forme recommandée ?	<code>foo <- function(x, ...) UseMethod("foo", x)</code>
R a-t-il un vrai système objet ?	Non — « <i>rather than a full-fledged ...</i> »
Qu'est-ce que l'attribut <code>class</code> ?	Un vecteur de caractères de noms de classes
Comment crée-t-on un objet de classe <code>"foo"</code> ?	En attachant un attribut <code>class</code>
Y a-t-il un contrôle ?	Aucun
Quel argument détermine la méthode ?	Le premier , et lui seul
Les classes des autres arguments ?	Ignorées
Ordre de recherche pour <code>c("foo", "bar")</code> ?	<code>mean.foo</code> → <code>mean.bar</code> → <code>mean.default</code>
Si rien n'est trouvé ?	Une erreur
Que recommande le manuel ?	Toujours écrire une méthode par défaut
Si l'objet n'a pas d'attribut <code>class</code> ?	<code>generic.default</code>
Comment appelle-t-on <code>mean.foo</code> ?	Une méthode
<code>UseMethod</code> crée-t-il un environnement ?	Non — il réutilise celui de la générique
Que deviennent les arguments ?	Ils sont réappariés aux formels de la méthode
<code>UseMethod</code> rend-il la main ?	Non, jamais
Conséquence ?	Le code après lui n'est jamais exécuté
D'où peut-on appeler <code>UseMethod</code> ?	Du corps d'une fonction seulement

Question	Réponse
Sans premier argument ?	Il prend le nom de la fonction courante
Avec deux arguments ?	Le second est l'objet de dispatch
Les quatre variables spéciales ?	<code>.Generic</code> · <code>.Class</code> · <code>.Method</code> · <code>.Group</code>
Que contient <code>.Class</code> ?	La classe de l'objet
Que contient <code>.Method</code> ?	Le nom de la méthode en cours
Qui contrôle les dispatchs suivants ?	Ces variables , pas l'objet
Comment se fait l'héritage ?	Par <code>NextMethod</code>
Que fait <code>NextMethod</code> ?	Il passe à la classe suivante du vecteur
Le patron d'usage ?	Faire quelques opérations, puis déléguer
Sa syntaxe complète ?	<code>NextMethod(generic, object, ...)</code>
Sans générique ?	Il prend <code>.Generic</code>
Sans objet ?	Le premier argument de la méthode courante
À quoi sert le <code>...</code> ?	À modifier les arguments du suivant
Les arguments non évalués ?	Restent non évalués
Les arguments manquants ?	Restent manquants
Que se passe-t-il si une méthode est appelée directement ?	Pas de <code>.Generic</code> ni <code>.Class</code>
Que faire alors ?	Toujours nommer le générique dans <code>NextMethod</code>
D'où vient <code>.Class</code> dans ce cas ?	De la classe du premier argument
Les trois groupes de méthodes ?	<code>Math</code> , <code>Summary</code> , <code>Ops</code>
Peut-on en ajouter ?	Non
Que contient <code>Summary</code> ?	<code>all any max min prod range sum</code>
Que contient <code>Ops</code> ?	Les opérateurs arithmétiques, de comparaison, logiques

Question	Réponse
Quand une méthode <code>Ops</code> est-elle invoquée ?	Si les deux opérandes suggèrent une méthode unique
Sinon ?	La méthode par défaut
Qui l'emporte, classe ou groupe ?	La classe
Combien d'éléments a <code>.Method</code> dans <code>Ops</code> ?	Deux
Que vaut l'élément non concerné ?	La chaîne vide <code>""</code>
Lister les méthodes d'une classe ?	<code>methods(class = "...")</code>
Lister les classes d'une générique ?	<code>methods(generique)</code>
Que signale un astérisque ?	Une méthode non visible par son nom
Comment la lire ?	<code>getS3method()</code> ou <code>getAnywhere()</code>
Pourquoi <code>get("gen.c1")</code> peut-il échouer ?	À cause des espaces de noms
Que fait <code>unclass()</code> ?	Retire temporairement l'effet de classe
Quel usage le cours lui donne-t-il ?	Apprendre la notion de classe et de générique
Que ne faut-il jamais faire côté nommage ?	Nommer une fonction <code>gen.c1</code> sans le vouloir
Combien de méthodes pour <code>print</code> ?	Environ 40
Quel est l'usage principal du système ?	L' affichage spécialisé des objets
Où vit le système S4 ?	Le paquet <code>methods</code>
Quelle recommandation générale du manuel ?	User du système de façon simple